

```

import csv

import re

from bs4 import BeautifulSoup

import requests

# Target URL (Using a dedicated public web scraping practice site)
URL = "https://realpython.github.io/fake-jobs/"

HEADERS = {
    "User-Agent": (
        "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
        "AppleWebKit/537.36 (KHTML, like Gecko) "
        "Chrome/120.0.0.0 Safari/537.36"
    )
}

def fetch_page_html(url):
    """Fetches the raw HTML content from the target URL."""
    try:
        response = requests.get(url, headers=HEADERS, timeout=10)
        response.raise_for_status()
        return response.text
    except requests.RequestException as error:
        print(f"[ERROR] Failed to retrieve web page: {error}")
        return None

def clean_text(text):
    """Cleans raw scraped text by stripping extra whitespace and newlines."""
    if not text:
        return "N/A"

    # Remove excessive spaces and newlines

```

```
cleaned = re.sub(r"\s+", " ", text).strip()
return cleaned if cleaned else "N/A"
```

```
def parse_property_listings(html):
    """Parses HTML content and extracts property information."""
    soup = BeautifulSoup(html, "html.parser")
    listings = []

    # Locate container elements (adjust selectors based on target website structure)
    cards = soup.find_all("div", class_="card-content")

    for card in cards:
        # Extract title, location, and price/subtitle
        raw_title = card.find("h2", class_="title")
        raw_location = card.find("p", class_="location")
        raw_price = card.find("h3", class_="subtitle")

        title = (
            clean_text(raw_title.text) if raw_title else "Property Listing"
        )
        location = (
            clean_text(raw_location.text) if raw_location else "Location N/A"
        )
        price = (
            clean_text(raw_price.text) if raw_price else "Price on Request"
        )

        listings.append(
            {"Title": title, "Price": price, "Location": location}
        )

    return listings
```

```

def save_to_csv(data, output_filename="properties.csv"):
    """Exports structured data to a clean CSV file."""
    if not data:
        print("[WARNING] No data found to write to CSV.")
        return

    headers = ["Title", "Price", "Location"]

    try:
        with open(output_filename, "w", newline="", encoding="utf-8") as file:
            writer = csv.DictWriter(file, fieldnames=headers)
            writer.writeheader()
            writer.writerows(data)
        print(
            f"[SUCCESS] Exported {len(data)} property listings to '{output_filename}'."
        )
    except IOError as error:
        print(f"[ERROR] Failed to save CSV file: {error}")

def main():
    print("Starting automated property scraper...")
    html_content = fetch_page_html(URL)

    if html_content:
        extracted_data = parse_property_listings(html_content)
        save_to_csv(extracted_data)

if __name__ == "__main__":
    main()

```